

■ PeerSmash

Placement & Technical Interview Preparation Guide

1. Elevator Pitch (How to Introduce PeerSmash)

Interviewer: "Tell me about your project PeerSmash."

Ideal 1-Minute Response:

*"PeerSmash is a high-performance, zero-server peer-to-peer (P2P) file-sharing web application built with **React, Node.js, Socket.IO, and WebRTC**.*

*Unlike traditional cloud storage platforms like Google Drive or WeTransfer—where files must be uploaded to a central server and then downloaded—PeerSmash streams file binary data **directly between two browser sessions** via WebRTC RTCDataChannel.*

*My Node.js server acts strictly as a lightweight signaling relay for SDP exchange and ICE candidate discovery. **Zero bytes of file data ever touch or store on the backend server**, ensuring 100% client privacy and unlimited transfer speeds capped only by user bandwidth. Key engineering highlights include custom **16KB chunking, backpressure flow control** using bufferedAmount thresholds to prevent memory crashes, multi-peer mesh signaling, and instant mobile pairing via QR codes."*

2. Core WebRTC & Networking Concepts

Q1: Why do you need a Node.js signaling server if WebRTC is Peer-to-Peer?

WebRTC enables direct browser-to-browser communication, but two browsers behind separate routers (NATs and firewalls) cannot automatically discover each other's IP addresses, ports, or media/data capabilities.

- **Signaling Server Role:** A lightweight WebSocket server (built using Socket.IO in server/src/index.js) is required to exchange SDP Offers/Answers and ICE Candidates between Peer A and Peer B.
- Once the direct WebRTC DataChannel connection is established (CONNECTED), the signaling server steps aside, and all file data flows purely peer-to-peer.

Q2: What is SDP (Session Description Protocol) and how does signaling work in PeerSmash?

SDP is a standardized string format that describes codec capabilities, network parameters, and protocol options of a peer.

Signaling Workflow in PeerSmash:

1. **Peer A (Initiator)** creates an `RTCPeerConnection` and calls `pc.createOffer()`.
2. Peer A sets its local description (`pc.setLocalDescription(offer)`) and emits it over Socket.IO: `socket.emit('offer', { roomId, targetId, offer })`.
3. The Node.js backend relays this offer to **Peer B** via `io.to(targetId).emit('offer')`.
4. **Peer B** receives the offer, sets `pc.setRemoteDescription(offer)`, calls `pc.createAnswer()`, sets its local description, and sends the answer back.
5. Peer A sets `pc.setRemoteDescription(answer)`. Both browsers are now network-aligned!

Q3: What are STUN, TURN, and ICE Candidates? Why are they needed?

- **NAT (Network Address Translation):** Routers hide private local IPs (e.g. 192.168.1.15) behind a single public IP.
- **STUN Server:** STUN (Session Traversal Utilities for NAT) tells a browser its public IP address and port visible to the public internet.
- **ICE (Interactive Connectivity Establishment):** Gathers candidate pathways (local IP, STUN public IP, TURN relay).
- **ICE Exchange:** As candidates are generated by `pc.onIceCandidate`, PeerSmash relays them via Socket.IO to the remote peer via `pc.addIceCandidate()`.

3. Deep-Dive: Backpressure & Memory Flow Control

Q4: How do you handle large files (e.g. 2 GB) in the browser without crashing the tab or RAM?

If you read a 2 GB file into memory at once or transmit faster than the WebRTC socket can send, the browser buffer overflows and crashes the tab. We solved this using **Chunking + Backpressure Control**:

1. **16KB Slicing:** In `fileChunker.js`, files are read in 16KB (16384 bytes) binary chunks using `file.slice(offset, offset + CHUNK_SIZE)` and `ArrayBuffer`.
2. **Backpressure Flow Control:** WebRTC DataChannels store unsent data in `dc.bufferedAmount`.
 - **High Water Mark (64 KB):** Before reading/sending the next chunk, we check if `dc.bufferedAmount > HIGH_WATER_MARK`.
 - **Pause & Drain:** If the buffer is full, the chunk loop pauses (`await waitForBufferDrain()`) until `dc.bufferedAmount` drains down to the **Low Water Mark (16 KB)** using `dc.bufferedAmountLowThreshold`.

Q5: How does the receiving side assemble incoming chunks into a downloadable file?

1. **Metadata Phase:** Sender first transmits a JSON string containing { type: 'file-meta', name, size, mimeType }. The FileReceiver initializes an empty array receivedChunks = [].
2. **Chunk Processing:** Raw ArrayBuffer binary chunks arriving on dc.onmessage are pushed directly into receivedChunks.
3. **Blob Assembly:** Upon receiving { type: 'file-complete' }, the receiver constructs a Blob:

```
const blob = new Blob(this.receivedChunks, { type: metadata.mimeType });
const url = URL.createObjectURL(blob);
```
4. **Auto-Download:** An invisible <a> link element is created programmatically, triggering a.click() to prompt browser download directly from memory URL.

4. Room State & React Architecture Questions

Q6: How is room state and mesh topology managed?

- PeerSmash features a **Full Mesh Topology** supporting up to **8 peers per room** (1 initiator + 7 joiners).
- In roomManager.js, room states are managed in-memory using ES6 Map and Set structures mapping 6-character room codes to peer socket IDs.
- **Mesh Connection:** When a new peer joins, existing peers create distinct RTCPeerConnection instances for the joiner. File streams are broadcast across active open DataChannels.

Q7: Why use useRef instead of useState for WebRTC connections and Socket instances?

In React, updating state triggers component re-renders. RTCPeerConnection, RTCDataChannel, and Socket instances are operational objects, not UI rendering state.

If stored in useState, re-renders would recreate listeners or reset connections. useRef preserves mutable operational state across renders without triggering unnecessary component re-renders.

5. Technical Architecture Cheat Sheet

Concept / Feature	Implementation in PeerSmash	Engineering Justification
Signaling Protocol	Socket.IO (WebSockets / HTTP fallback)	Low-latency bi-directional SDP event relay
P2P Transfer Channel	WebRTC RTCDataChannel (ordered)	Direct browser-to-browser TCP-like binary delivery
Chunk Size	16 KB (16,384 Bytes)	Fits within standard WebRTC DataChannel packet limits
Backpressure Control	64KB High / 16KB Low Watermark	Prevents browser socket buffer overflow & RAM crash
NAT Traversal	Google & Twilio STUN Servers	Discovers public IP and port mappings across NATs
Room Generator	6-Char Alphanumeric Code (in-memory Map)	Human-readable, collision-free room code

6. HR & Behavioral / Engineering Scenarios

Q8: What was the toughest technical challenge you faced, and how did you resolve it?

Response: "The hardest challenge was handling **race conditions during WebRTC signaling and ICE candidate timing**.

Early in development, ICE candidates were generated by the initiating peer before the receiving peer had set remoteDescription. Calling pc.addIceCandidate() prematurely caused connection failures.

Solution: I engineered an **ICE candidate buffer queue** (iceCandidatesQueueRef). If a candidate arrives before remoteDescription is set, it is buffered in a Map. As soon as setRemoteDescription() completes, the queue automatically drains and applies all buffered candidates sequentially."

Q9: How does your app scale? What if 100,000 users join simultaneously?

Response: "Because PeerSmash is a **P2P architecture**, central server bandwidth cost is almost zero! The backend server only handles lightweight JSON signaling messages (a few KB per room). 100% of gigabyte file data flows directly between client browsers. To scale the signaling backend to millions of rooms, we can scale Node.js instances horizontally behind a load balancer using a Redis Pub/Sub adapter for Socket.IO."

Q10: What happens if two devices are behind strict corporate firewalls (Symmetric NAT)?

Response: "STUN alone cannot connect peers behind strict Symmetric NATs because port mappings vary per destination host. In production enterprise environments, WebRTC falls back to **TURN (Traversal Using Relays around NAT)** servers. Adding a Coturn TURN server cluster ensures 99.99% connection success rate across corporate networks."